pytorch - nn

Lecture 25

Dr. Colin Rundel

Odds & Ends

Torch models

Implementation details:

- Models are implemented as a class inheriting from `torch.nn.Module`
- Must implement constructor and `forward()` method
  - `__init__()` should call parent constructor via `super()`
    - Use `torch.nn.Parameter()` to indicate model parameters
  - `forward()` should implement the model - constants + parameters -> `return` predictions

Fitting procedure:

- For each iteration of solver:
  - Get current predictions via a call to `forward()` or equivalent.
  - Calculate a (scalar) loss or equivalent
  - Call `backward()` method on loss
  - Use built-in optimizer (`step()` and then `zero_grad()` if necessary)

From last time

```python
class Model(torch.nn.Module):
    def __init__(self, X, y, beta=None):
        super().__init__()
        self.X = X
        self.y = y
        if beta is None:
          beta = torch.zeros(X.shape[1])
        beta.requires_grad = True
        self.beta = torch.nn.Parameter(beta)

    def forward(self, X):
        return X @ self.beta

    def fit(self, opt, n=1000, loss_fn = torch.nn.MSELoss()):
      losses = []
      for i in range(n):
          loss = loss_fn(
            self(self.X).squeeze(),
            self.y.squeeze()
```

What is `self(self.X)`?

This is (mostly) just short hand for calling `self.forward(X)` to generate the output tensor from the current values of the parameters.

This is done via the `__call__()` method in the `torch.nn.Module` class.

`__call__()` allows Python classes to be invoked like functions.

```python
class greet:
  def __init__(self, greeting):
    self.greeting = greeting
  def __call__(self, name):
    return self.greeting + " " + name
```

```python
hello = greet("Hello")
hello("Jane")
```

```
'Hello Jane'
```

```python
gm = greet("Good morning")
gm("Bob")
```

```
'Good morning Bob'
```

MNIST & Logistic models

MNIST handwritten digits - simplified

```python
from sklearn.datasets import load_digits
digits = load_digits()
```

```python
X = digits.data
X.shape
```

```
(1797, 64)
```

```python
y = digits.target
y.shape
```

```
(1797,)
```

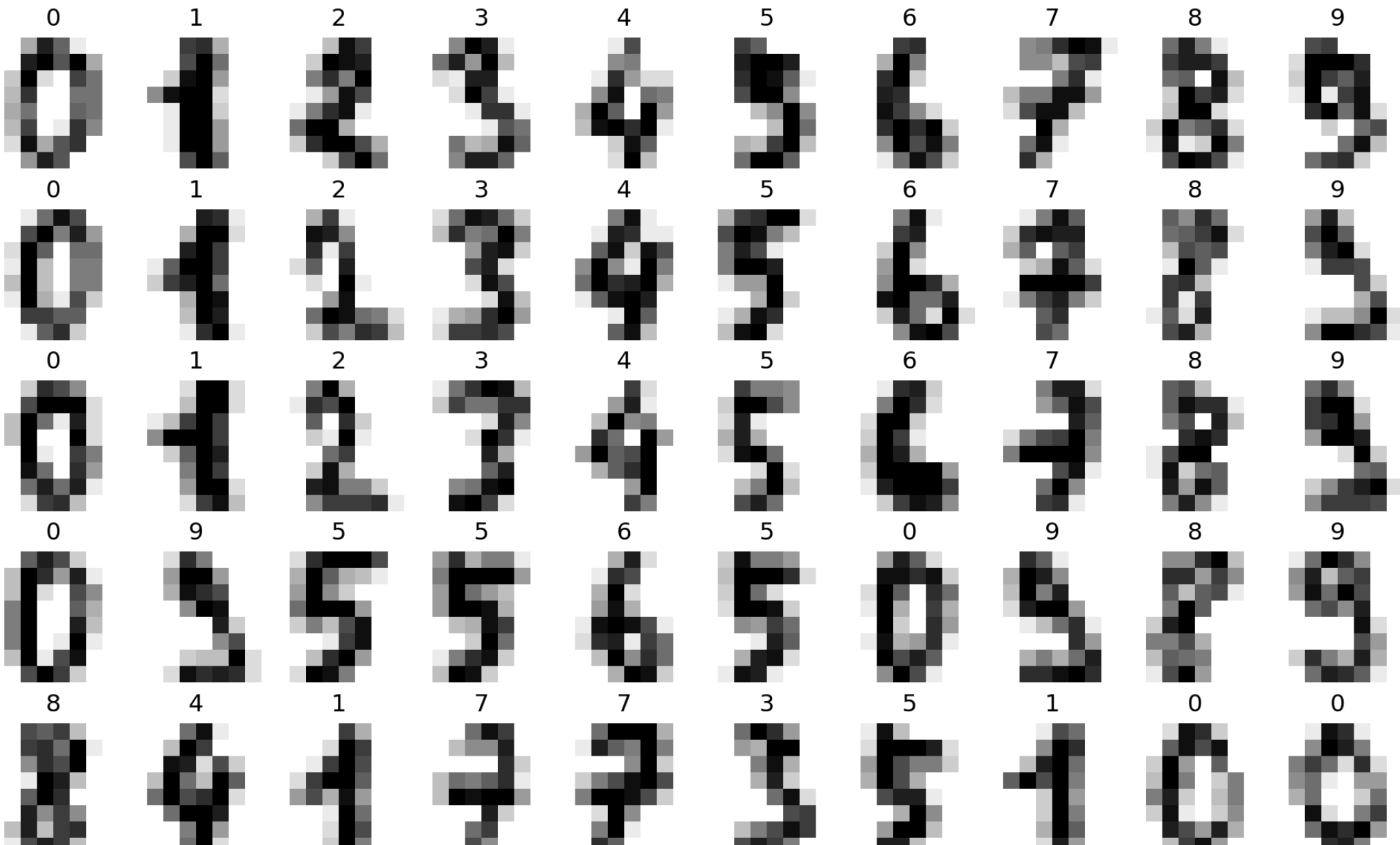
```python
X[0:2]
```

```
array([[ 0.,  0.,  5., 13.,  9.,  1.,  0.,
         0.,  0.,  0., 13., 15., 10., 15.,
         5.,  0.,  0.,  3., 15.,  2.,  0.,
        11.,  8.,  0.,  0.,  4., 12.,  0.,
         0.,  8.,  8.,  0.,  0.,  5.,  8.,
         0.,  0.,  9.,  8.,  0.,  0.,  4.,
        11.,  0.,  1., 12.,  7.,  0.,  0.,
         2., 14.,  5., 10., 12.,  0.,  0.,
         0.,  0.,  6., 13., 10.,  0.,  0.,
         0.],
       [ 0.,  0.,  0., 12., 13.,  5.,  0.,
         0.,  0.,  0.,  0., 11., 16.,  9.,
         0.,  0.,  0.,  0.,  3., 15., 16.,
         6.,  0.,  0.,  0.,  7., 15., 16.,
        16.,  2.,  0.,  0.,  0.,  0.,  1.,
        16., 16.,  3.,  0.,  0.,  0.,  0.,
         1., 16., 16.,  6.,  0.,  0.,  0.,
         0.,  1., 16., 16.,  6.,  0.,  0.,
         0.,  0.,  0., 11., 16., 10.,  0.,
         0.]])
```

```python
y[0:10]
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Example digits



Test train split

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, shuffle=True, random_state=1234
)
```

```python
X_train.shape
```

```
(1437, 64)
```

```python
y_train.shape
```

```
(1437,)
```

```python
X_test.shape
```

```
(360, 64)
```

```python
y_test.shape
```

```
(360,)
```

```python
from sklearn.linear_model import LogisticRegress
from sklearn.metrics import accuracy_score
```

```python
lr = LogisticRegression(
  penalty=None
).fit(
  X_train, y_train
)
```

```python
accuracy_score(y_train, lr.predict(X_train))
```

```
1.0
```

```python
accuracy_score(y_test, lr.predict(X_test))
```

```
0.9583333333333334
```

As Torch tensors

```
X_train = torch.from_numpy(X_train).float()
y_train = torch.from_numpy(y_train)
X_test = torch.from_numpy(X_test).float()
y_test = torch.from_numpy(y_test)
```

```
X_train.shape
```

```
torch.Size([1437, 64])
```

```
X_train.dtype
```

```
torch.float32
```

```
y_train.shape
```

```
torch.Size([1437])
```

```
y_train.dtype
```

```
torch.int64
```

```
X_test.shape
```

```
torch.Size([360, 64])
```

```
X_test.dtype
```

```
torch.float32
```

```
y_test.shape
```

```
torch.Size([360])
```

```
y_test.dtype
```

```
torch.int64
```

PyTorch Model

```python
class mnist_model(torch.nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.beta = torch.nn.Parameter(
          torch.randn(input_dim, output_dim, requires_grad=True)
        )
        self.intercept = torch.nn.Parameter(
          torch.randn(output_dim, requires_grad=True)
        )

    def forward(self, X):
        return (X @ self.beta + self.intercept).squeeze()

    def fit(self, X_train, y_train, X_test, y_test, lr=0.001, n=1000):
      opt = torch.optim.SGD(self.parameters(), lr=lr, momentum=0.9)
      losses = []

      for i in range(n):
          opt.zero_grad()
          loss = torch.nn.CrossEntropyLoss()(self(X_train), y_train)
          loss.backward()
          opt.step()
```

Cross entropy loss

From the pytorch documentation:

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = -w_{y_n} \log \frac{\exp\left(x_{n,y_n}\right)}{\sum_{c=1}^{C} \exp\left(x_{n,c}\right)} 1\{y_n \neq \text{ ignore_index }\}$$

which is then aggregated across the N observations according to the `reduction` argument:

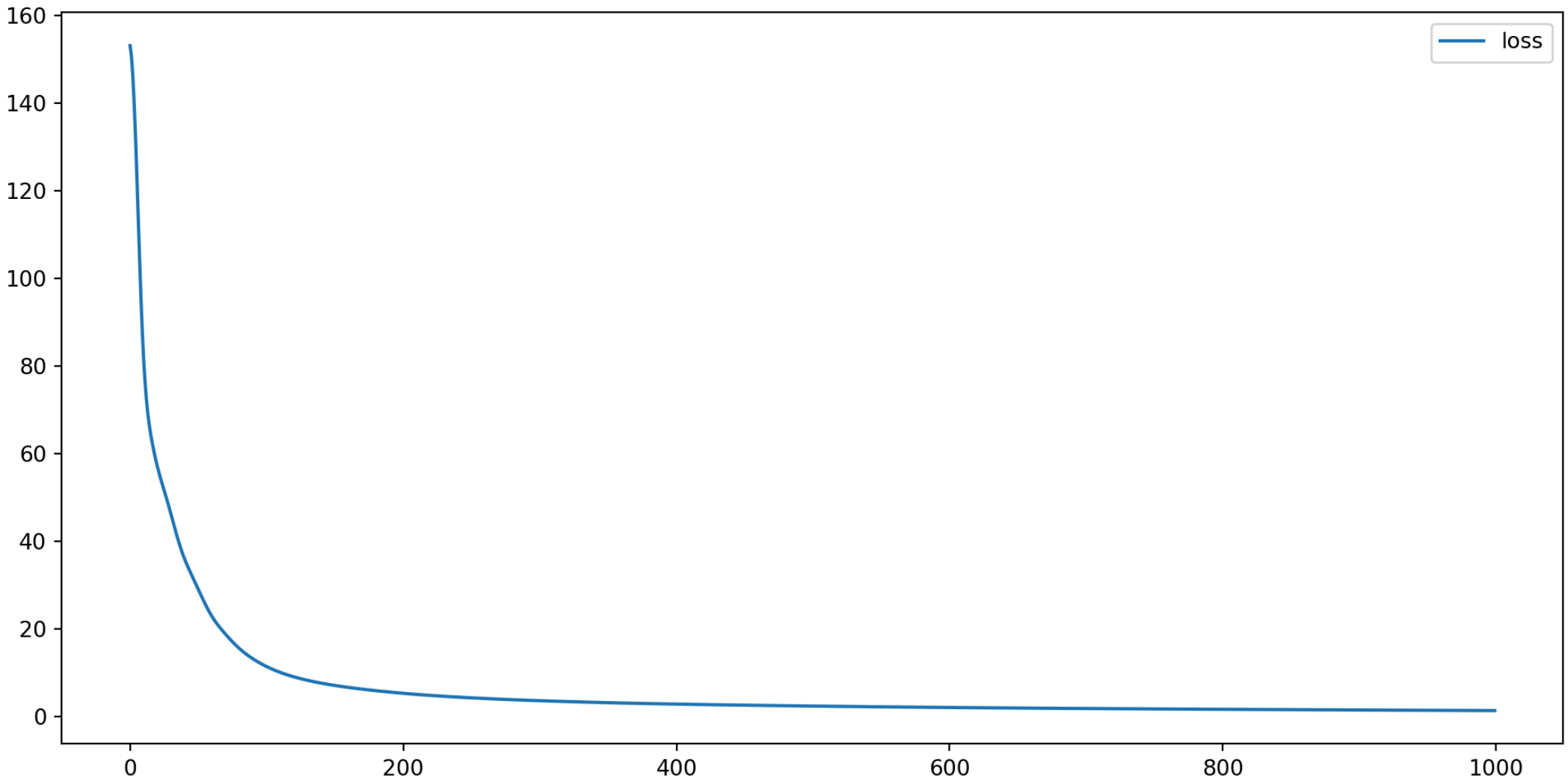
$$\ell(x, y) = \begin{cases} \sum_{n=1}^{N} \frac{1}{\sum_{n=1}^{N} w_{y_n} \cdot 1\{y_n \neq \text{ ignore_index }\}} l_n, & \text{if reduction = 'mean'} \\ \sum_{n=1}^{N} l_n, & \text{if reduction = 'sum'} \end{cases}$$

This is just the negative log-likelihood of the multinomial regression model,

$$L(y_n = c \mid x_n) = \frac{\exp(x_n^\top \beta_c)}{\sum_{k=1}^{C} \exp(x_n^\top \beta_k)}, \quad c = 1, \dots, C$$

Cross entropy loss

```
model = mnist_model(64, 10)
l = model.fit(X_train, y_train, X_test, y_test)
```



Out-of-sample accuracy

```python
model(X_test)
```

```
tensor([[-3.0381e+01, -4.2913e+01,
          2.9547e+01,  7.0642e+00,
         -1.7528e+01, -1.2911e+01,
         -3.5493e+01,  6.4838e+01,
         -6.3084e+00, -4.1427e+01],
        [ 1.6111e+01,  5.9402e+01,
         -9.7221e+01, -1.8355e+01,
          2.2700e+01,  1.7079e+01,
         -2.7102e+01,  1.5978e+01,
          1.1134e+01,  4.3695e+01],
        [ 3.8016e+00, -2.9081e+01,
          2.8983e+01,  4.0087e+01,
          7.8980e+00, -3.1791e+01,
         -3.6101e+01,  1.1112e+02,
          2.8810e+01, -1.7109e+00],
        [ 1.6034e+01, -5.2138e+00,
         -5.6606e+00, -1.6190e+01,
          1.5731e+01, -1.9501e+01,
          7.1145e+01, -9.5330e+00,
```

```python
value, index = torch.max(model(X_test), dim=1)
index
```

```
tensor([7, 1, 7, 6, 0, 2, 4, 3, 6, 3, 7, 8, 7,
        9, 4, 3, 6, 7, 8, 4, 0, 3, 9, 1, 3, 6,
        6, 0, 5, 4, 1, 2, 1, 2, 3, 2, 7, 6, 4,
        8, 6, 4, 4, 0, 9, 7, 8, 5, 4, 4, 4, 1,
        7, 6, 8, 2, 9, 9, 8, 0, 1, 3, 1, 8, 8,
        1, 3, 9, 1, 3, 9, 6, 9, 5, 2, 1, 9, 2,
        1, 3, 8, 7, 3, 3, 8, 7, 7, 5, 1, 2, 6,
        7, 9, 1, 6, 4, 5, 2, 2, 4, 5, 4, 7, 6,
        5, 9, 2, 4, 1, 0, 7, 6, 1, 2, 9, 5, 2,
        5, 0, 3, 2, 7, 6, 4, 3, 2, 1, 1, 6, 7,
        6, 2, 3, 4, 7, 5, 0, 9, 1, 0, 5, 6, 7,
        6, 3, 8, 3, 2, 0, 4, 6, 9, 5, 4, 6, 1,
        4, 1, 6, 2, 7, 9, 0, 4, 9, 5, 4, 1, 3,
        8, 6, 4, 7, 1, 5, 7, 4, 7, 4, 3, 2, 2,
        1, 1, 4, 4, 3, 5, 5, 9, 4, 5, 5, 9, 3,
        9, 0, 1, 2, 0, 8, 2, 8, 5, 2, 4, 6, 8,
        3, 8, 1, 0, 8, 1, 8, 5, 6, 8, 7, 1, 8,
        0, 4, 0, 7, 0, 5, 5, 6, 1, 3, 0, 5, 8,
        8, 0, 9, 8, 6, 7, 8, 4, 1, 0, 5, 1, 5,
```

```python
(index == y_test).sum()
```

```
tensor(326)
```

```python
(index == y_test).sum() / len(y_test)
```

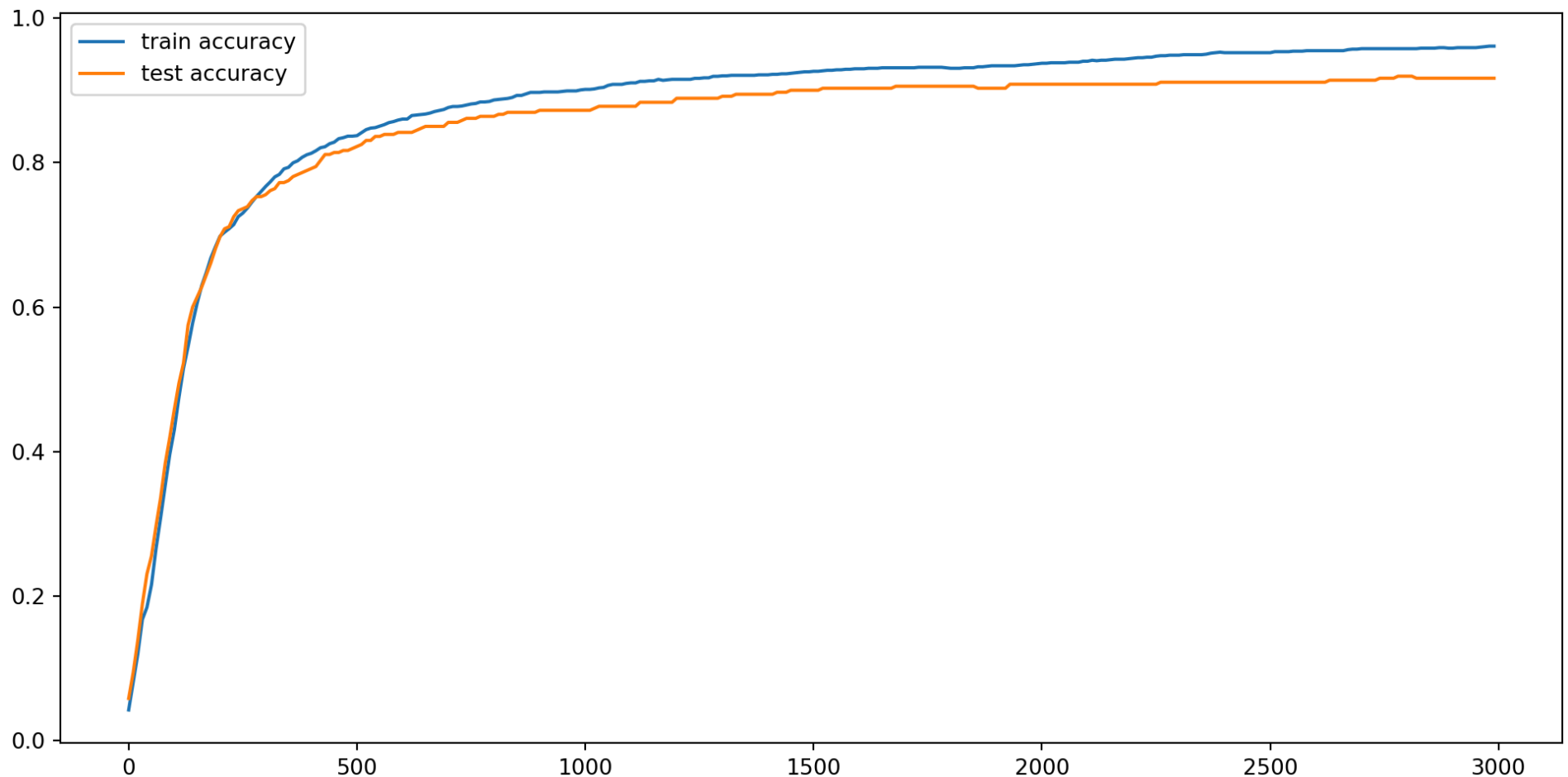
```
tensor(0.9056)
```

Calculating Accuracy

```python
class mnist_model(torch.nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.beta = torch.nn.Parameter(
          torch.randn(input_dim, output_dim, requires_grad=True)
        )
        self.intercept = torch.nn.Parameter(
          torch.randn(output_dim, requires_grad=True)
        )

    def forward(self, X):
        return (X @ self.beta + self.intercept).squeeze()

    def fit(self, X_train, y_train, X_test, y_test, lr=0.001, n=1000, acc_step=10):
      opt = torch.optim.SGD(self.parameters(), lr=lr, momentum=0.9)
      losses, train_acc, test_acc = [], [], []

      for i in range(n):
          opt.zero_grad()
          loss = torch.nn.CrossEntropyLoss()(self(X_train), y_train)
          loss.backward()
          opt.step()
          losses.append(loss.item())

```

Performance

```
loss, train_acc, test_acc = mnist_model(
  64, 10
).fit(
  X_train, y_train, X_test, y_test, acc_step=10, n=3000
)
```



NN Layers

```python
class mnist_nn_model(torch.nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.linear = torch.nn.Linear(input_dim, output_dim)

    def forward(self, X):
        return self.linear(X)

    def fit(self, X_train, y_train, X_test, y_test, lr=0.001, n=1000, acc_step=10):
      opt = torch.optim.SGD(self.parameters(), lr=lr, momentum=0.9)
      losses, train_acc, test_acc = [], [], []

      for i in range(n):
          opt.zero_grad()
          loss = torch.nn.CrossEntropyLoss()(self(X_train), y_train)
          loss.backward()
          opt.step()
          losses.append(loss.item())

          if (i+1) % acc_step == 0:
            val, train_pred = torch.max(self(X_train), dim=1)
            val, test_pred  = torch.max(self(X_test), dim=1)
```

NN linear layer

Applies a linear transform to the incoming data (X):

$$y = XA^T + b$$

```
X.shape
```

```
(1797, 64)
```

```
model = mnist_nn_model(64, 10)
model.parameters()
```

```
<generator object Module.parameters at 0x34efa6200>
```

```
list(model.parameters())[0].shape  # A - weights (betas)
```

```
torch.Size([10, 64])
```

```
list(model.parameters())[1].shape  # b - bias (intercept)
```

```
torch.Size([10])
```

Performance

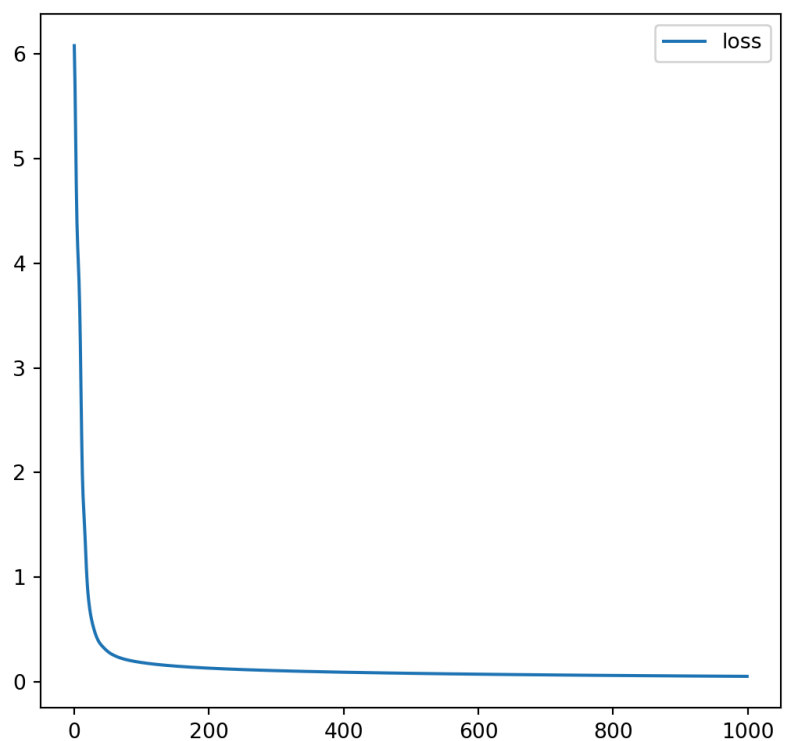
```
loss, train_acc, test_acc = model.fit(X_train, y_train, X_test, y_test, n=1000)
```

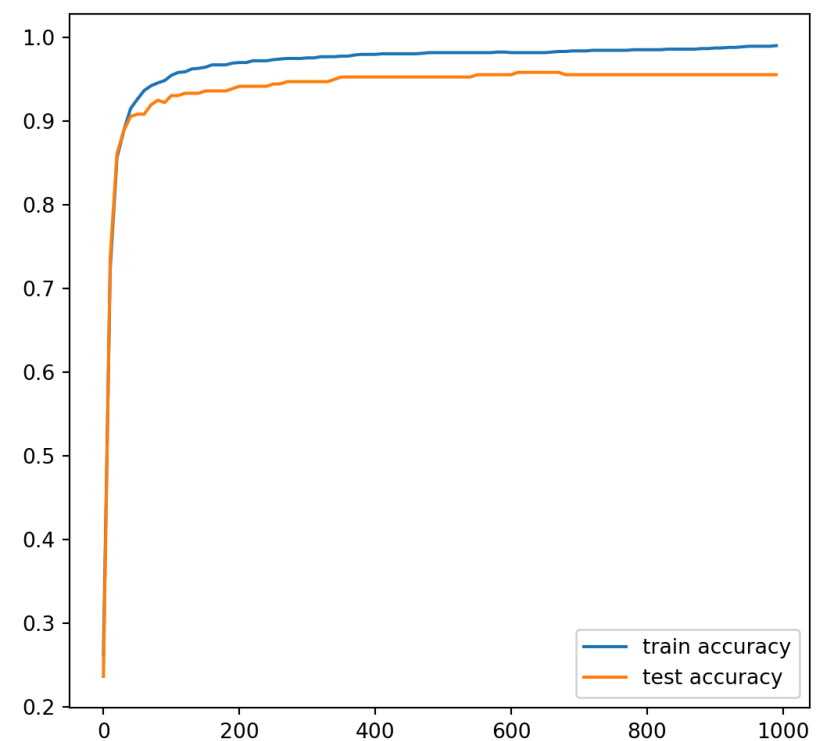
```
train_acc[-5:]
```

```
[tensor(0.9896), tensor(0.9896), tensor(0.9896),
```

```
test_acc[-5:]
```

```
[tensor(0.9556), tensor(0.9556), tensor(0.9556),
```





Feedforward Neural Network

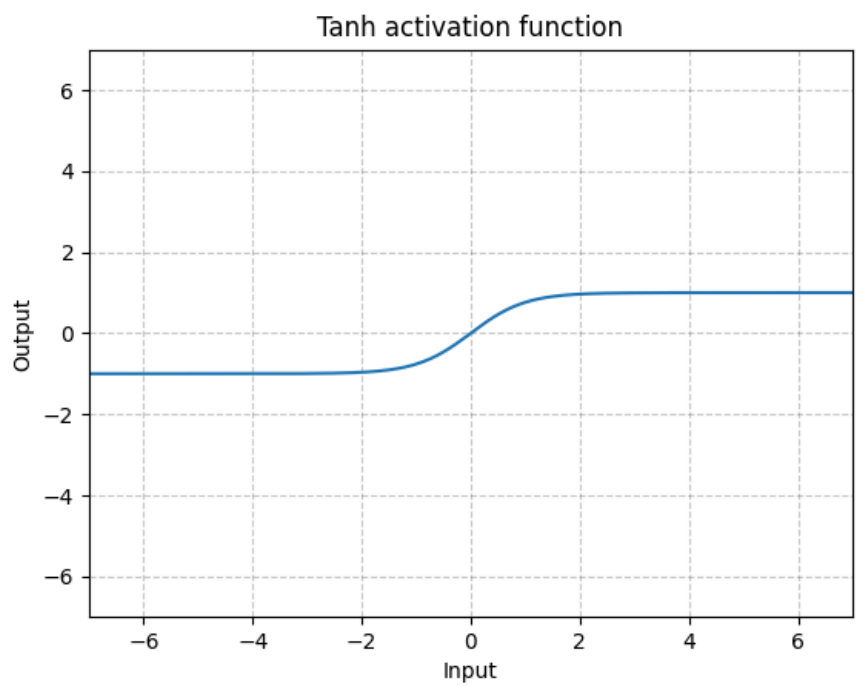
FNN Model

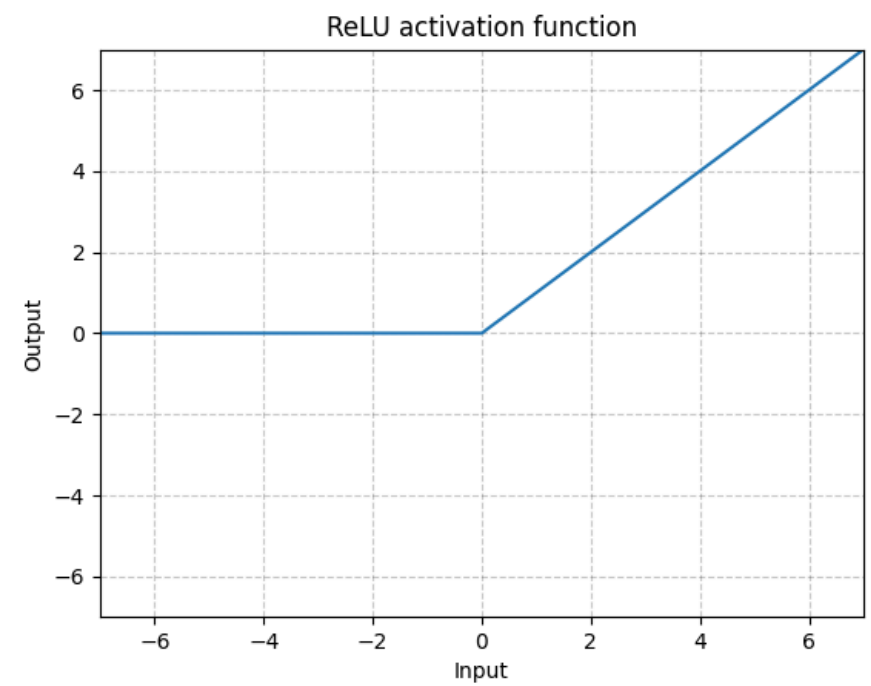
```
class mnist_fnn_model(torch.nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, nl_step = torch.nn.ReLU()):
        super().__init__()
        self.l1 = torch.nn.Linear(input_dim, hidden_dim)
        self.nl = nl_step
        self.l2 = torch.nn.Linear(hidden_dim, output_dim)

    def forward(self, X):
        out = self.l1(X)
        out = self.nl(out)
        out = self.l2(out)
        return out

    def fit(self, X_train, y_train, X_test, y_test, lr=0.001, n=1000, acc_step=10):
      opt = torch.optim.SGD(self.parameters(), lr=lr, momentum=0.9)
      losses, train_acc, test_acc = [], [], []

      for i in range(n):
          opt.zero_grad()
          loss = torch.nn.CrossEntropyLoss()(self(X_train), y_train)
          loss.backward()
          opt.step()
```

Non-linear activation functions

$$\mathrm{Tanh}(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

$$\mathrm{ReLU}(x) = \max(0, x)$$





Model parameters

```
model = mnist_fnn_model(64,64,10)
len(list(model.parameters()))
```

4

```
for i, p in enumerate(model.parameters()):
  print("Param", i, p.shape)
```

```
Param 0 torch.Size([64, 64])
Param 1 torch.Size([64])
Param 2 torch.Size([10, 64])
Param 3 torch.Size([10])
```

Performance - ReLU

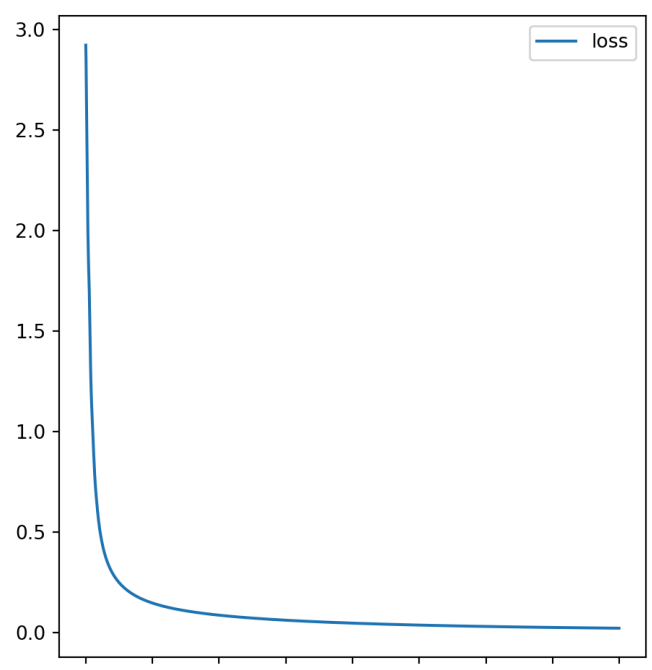
```
loss, train_acc, test_acc = mnist_fnn_model(64,64,10).fit(
  X_train, y_train, X_test, y_test, n=2000
)
```

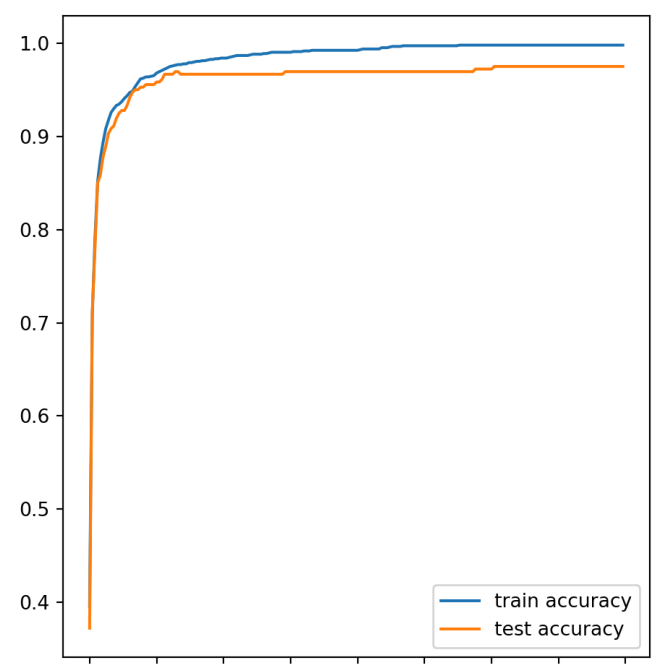
```
train_acc[-5:]
```

```
[0.9979123173277662, 0.9979123173277662, 0.99791
```

```
test_acc[-5:]
```

```
[0.975, 0.975, 0.975, 0.975, 0.975]
```





Performance - tanh

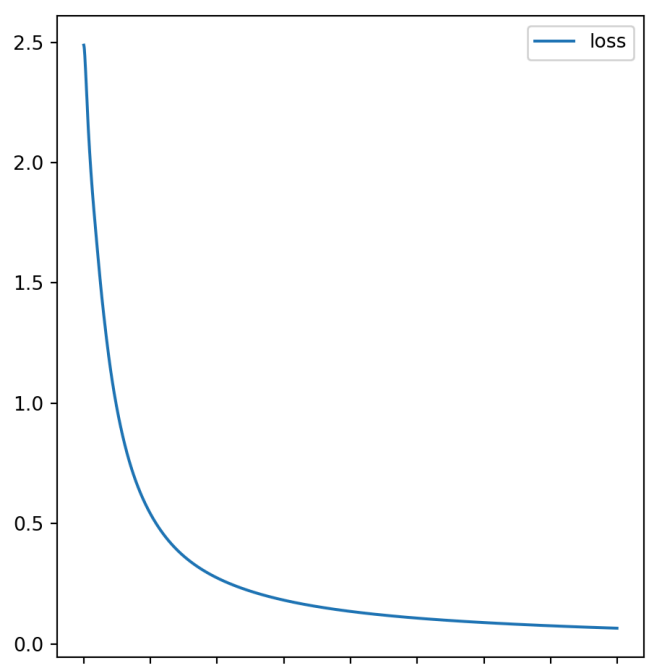
```
loss, train_acc, test_acc = mnist_fnn_model(64,64,10, nl_step=torch.nn.Tanh()).fit(
  X_train, y_train, X_test, y_test, n=2000
)
```

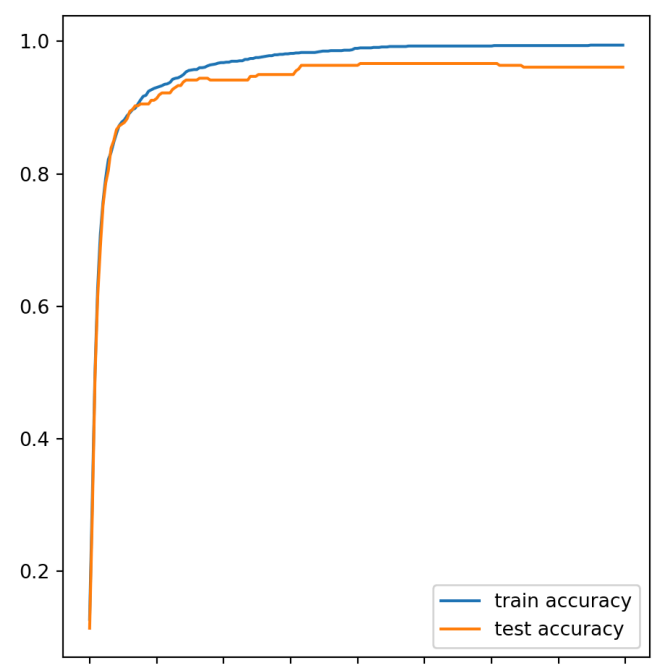
```
train_acc[-5:]
```

```
[0.9944328462073765, 0.9944328462073765, 0.99443
```

```
test_acc[-5:]
```

```
[0.9611111111111111, 0.9611111111111111, 0.96111
```





Adding another layer

```python
class mnist_fnn2_model(torch.nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, nl_step = torch.nn.ReLU()):
        super().__init__()
        self.l1 = torch.nn.Linear(input_dim, hidden_dim)
        self.nl1 = nl_step
        self.l2 = torch.nn.Linear(hidden_dim, hidden_dim)
        self.nl2 = nl_step
        self.l3 = torch.nn.Linear(hidden_dim, output_dim)

    def forward(self, X):
        out = self.l1(X)
        out = self.nl1(out)
        out = self.l2(out)
        out = self.nl2(out)
        out = self.l3(out)
        return out

    def fit(self, X_train, y_train, X_test, y_test, lr=0.001, n=1000, acc_step=10):
      loss_fn = torch.nn.CrossEntropyLoss()
      opt = torch.optim.SGD(self.parameters(), lr=lr, momentum=0.9)
      losses, train_acc, test_acc = [], [], []

```

Performance - relu

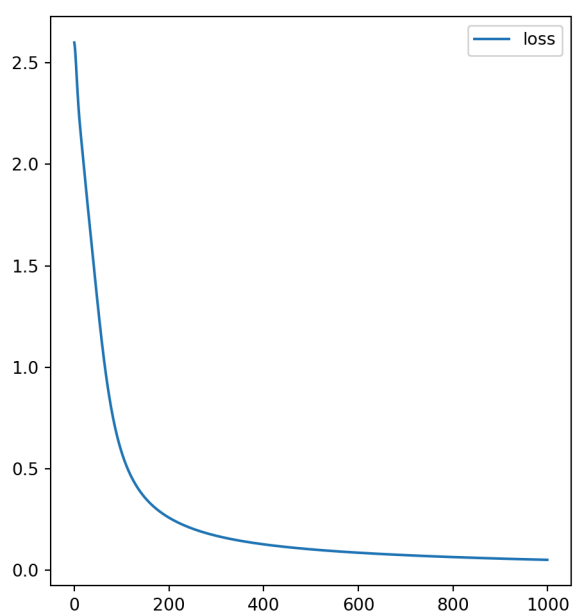
```
loss, train_acc, test_acc = mnist_fnn2_model(
  64,64,10, nl_step=torch.nn.ReLU()
).fit(
  X_train, y_train, X_test, y_test, n=1000
)
```

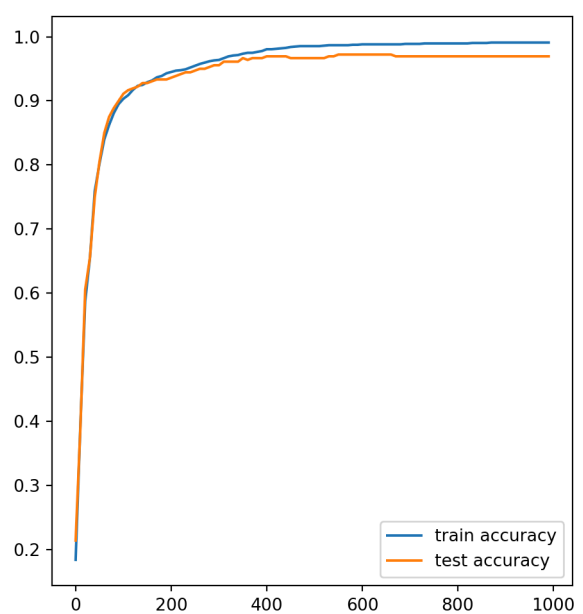
```
train_acc[-5:]
```

```
[0.9909533750869868, 0.9909533750869868, 0.99095
```

```
test_acc[-5:]
```

```
[0.9694444444444444, 0.9694444444444444, 0.96944
```





Performance - tanh

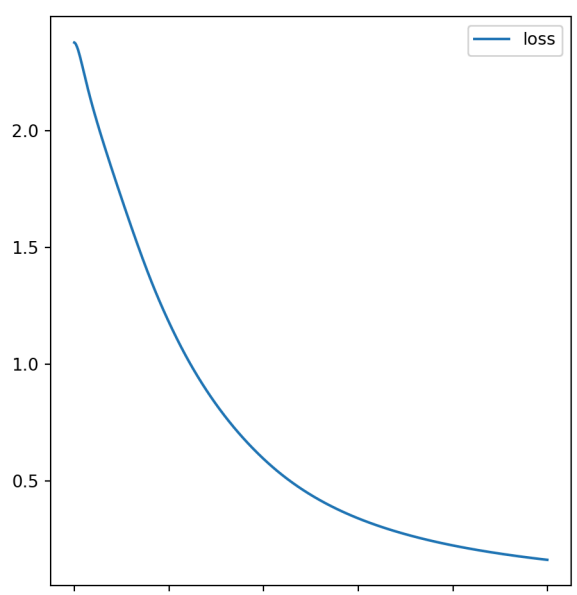
```
loss, train_acc, test_acc = mnist_fnn2_model(
  64,64,10, nl_step=torch.nn.Tanh()
).fit(
  X_train, y_train, X_test, y_test, n=1000
)
```

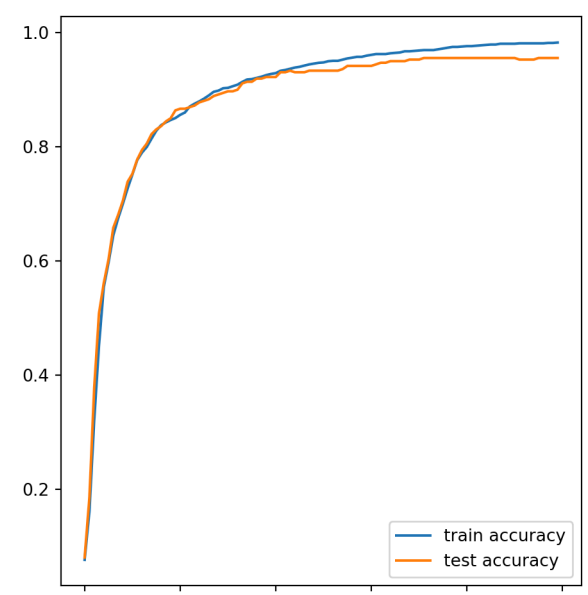
```
train_acc[-5:]
```

```
[0.9812108559498957, 0.9812108559498957, 0.98190
```

```
test_acc[-5:]
```

```
[0.9555555555555556, 0.9555555555555556, 0.95555
```



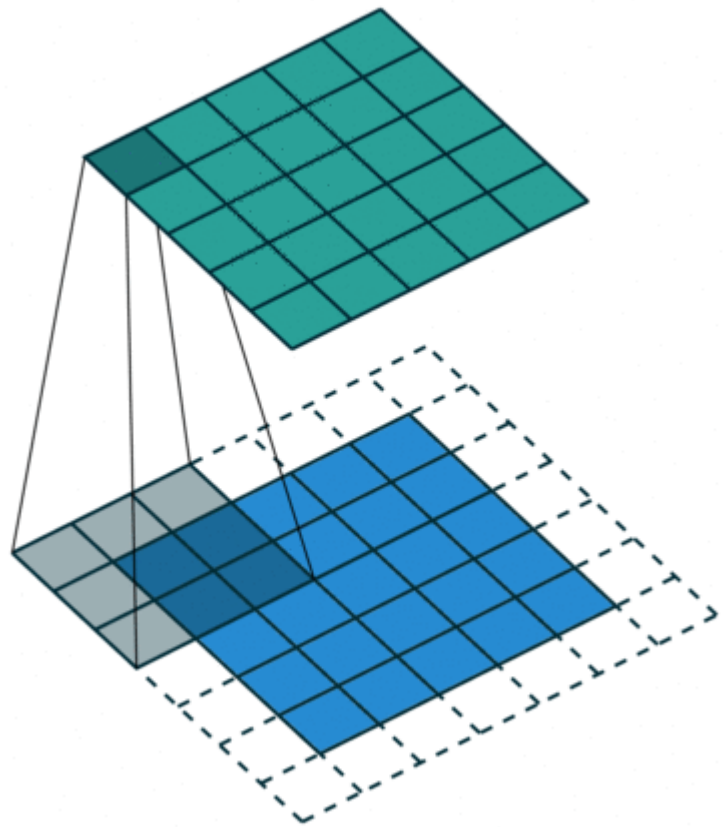


Convolutional NN

2d convolutions

3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0



nn.Conv2d()

```python
cv = torch.nn.Conv2d(
  in_channels=1, out_channels=4,
  kernel_size=3,
  stride=1, padding=1
)
```

```python
list(cv.parameters())[0] # kernel weights
```

```
Parameter containing:
tensor([[[[ 0.1937,  0.1034, -0.1968],
          [-0.2106, -0.1063, -0.1987],
          [-0.1544, -0.0823,  0.1624]]],

        [[[ 0.3281, -0.2475,  0.2985],
          [-0.0837, -0.2276,  0.0987],
          [-0.0877, -0.0379,  0.1052]]],

        [[[-0.2812,  0.1991, -0.0239],
          [-0.3327, -0.2178, -0.0093],
          [-0.3192,  0.2740, -0.1323]]],

        [[[-0.1357,  0.2482,  0.1384],
          [-0.1791,  0.0027, -0.2837],
          [-0.2920, -0.2582,  0.1475]]]],
       requires_grad=True)
```

```python
list(cv.parameters())[1] # biases
```

```
Parameter containing:
tensor([ 0.1338, -0.2994,  0.0742, -0.0760],
       requires_grad=True)
```

Applying `Conv2d()`

```python
X_train[[0]]
```

```
tensor([[ 0.,  0.,  0., 10., 11.,  0.,  0.,
          0.,  0.,  0.,  9., 16.,  6.,  0.,
          0.,  0.,  0.,  0., 15., 13.,  0.,
          0.,  0.,  0.,  0.,  0., 14., 10.,
          0.,  0.,  0.,  0.,  0.,  1., 15.,
         12.,  8.,  2.,  0.,  0.,  0.,  0.,
         12., 16., 16., 16., 10.,  1.,  0.,
          0.,  7., 16., 12., 12., 16.,  4.,
          0.,  0.,  0.,  9., 15., 12.,  5.,
          0.]])
```

```python
X_train[[0]].shape
```

```
torch.Size([1, 64])
```

```python
cv(X_train[[0]])
```

```
RuntimeError: Expected 3D (unbatched) or 4D (batched)
```

```python
X_train[[0]].view(1,8,8)
```

```
tensor([[[ 0.,  0.,  0., 10., 11.,  0.,  0.,
           0.],
         [ 0.,  0.,  9., 16.,  6.,  0.,  0.,
           0.],
         [ 0.,  0., 15., 13.,  0.,  0.,  0.,
           0.],
         [ 0.,  0., 14., 10.,  0.,  0.,  0.,
           0.],
         [ 0.,  1., 15., 12.,  8.,  2.,  0.,
           0.],
         [ 0.,  0., 12., 16., 16., 16., 10.,
           1.],
         [ 0.,  0.,  7., 16., 12., 12., 16.,
           4.],
         [ 0.,  0.,  0.,  9., 15., 12.,  5.,
           0.]]])
```

```python
cv(X_train[[0]].view(1,8,8))
```

```
tensor([[[ 1.3375e-01,  1.5954e+00,
           4.1337e-03, -4.8475e+00,
          -6.1056e+00, -3.1089e+00,
           1.3375e-01,  1.3375e-01],
         [ 1.3375e-01,  7.8146e-01,
          -5.0938e+00, -9.1715e+00,
          -2.8067e+00,  1.0006e+00,
           1.3375e-01,  1.3375e-01],
         [ 1.3375e-01, -2.3441e+00,
          -5.7907e+00, -5.1754e+00,
          -4.2875e-01,  1.2957e+00,
           1.3375e-01,  1.3375e-01],
         [ 2.9616e-01, -3.2460e+00,
          -3.7897e+00, -1.6332e+00,
          -1.6409e+00, -1.2660e+00,
          -1.7501e-01,  1.3375e-01],
```

Pooling

```python
x = torch.tensor(
  [[[0,0,0,0],
    [0,1,2,0],
    [0,3,4,0],
    [0,0,0,0]]],
  dtype=torch.float
)
x.shape
```

```
torch.Size([1, 4, 4])
```

```python
torch.nn.MaxPool2d(
  kernel_size=2, stride=1
)(x)
```

```
tensor([[[1., 2., 2.],
         [3., 4., 4.],
         [3., 4., 4.]]])
```

```python
torch.nn.MaxPool2d(
  kernel_size=3, stride=1, padding=1
)(x)
```

```
tensor([[[1., 2., 2., 2.],
         [3., 4., 4., 4.],
         [3., 4., 4., 4.],
         [3., 4., 4., 4.]]])
```

```python
torch.nn.AvgPool2d(
  kernel_size=2
)(x)
```

```
tensor([[[0.2500, 0.5000],
         [0.7500, 1.0000]]])
```

```python
torch.nn.AvgPool2d(
  kernel_size=2, padding=1
)(x)
```

```
tensor([[[0.0000, 0.0000, 0.0000],
         [0.0000, 2.5000, 0.0000],
         [0.0000, 0.0000, 0.0000]]])
```

Convolutional model

```python
class mnist_conv_model(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.cnn  = torch.nn.Conv2d(
          in_channels=1, out_channels=8,
          kernel_size=3, stride=1, padding=1
        )
        self.relu = torch.nn.ReLU()
        self.pool = torch.nn.MaxPool2d(kernel_size=2)
        self.lin  = torch.nn.Linear(8 * 4 * 4, 10)

    def forward(self, X):
        out = self.cnn(X.view(-1, 1, 8, 8))     # (N, 1, 8, 8) -> (N, 8, 8, 8)
        out = self.relu(out)                    # (N, 8, 8, 8)
        out = self.pool(out)                    # (N, 8, 8, 8) -> (N, 8, 4, 4)
        out = self.lin(out.view(-1, 8 * 4 * 4)) # (N, 128) -> (N, 10)
        return out

    def fit(self, X_train, y_train, X_test, y_test, lr=0.001, n=1000, acc_step=10):
      loss_fn = torch.nn.CrossEntropyLoss()
      opt = torch.optim.SGD(self.parameters(), lr=lr, momentum=0.9)
      losses, train_acc, test_acc = [], [], []
```

Performance

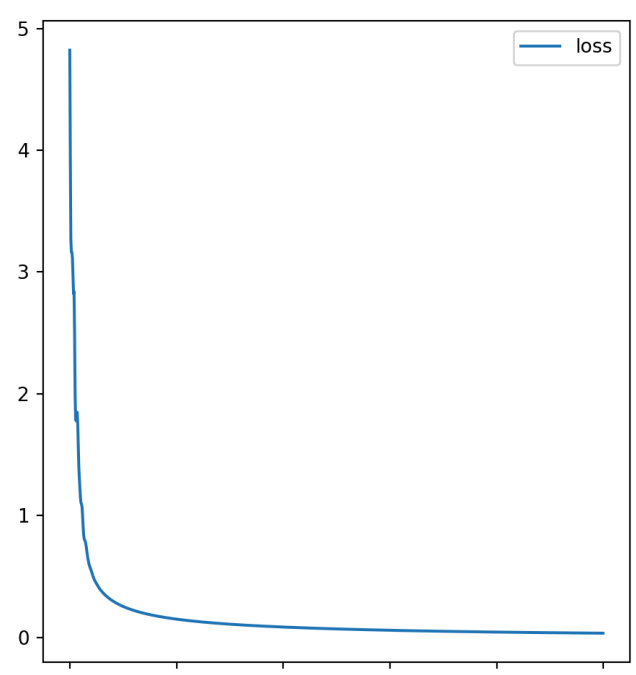
```
loss, train_acc, test_acc = mnist_conv_model().fit(
  X_train, y_train, X_test, y_test, n=1000
)
```

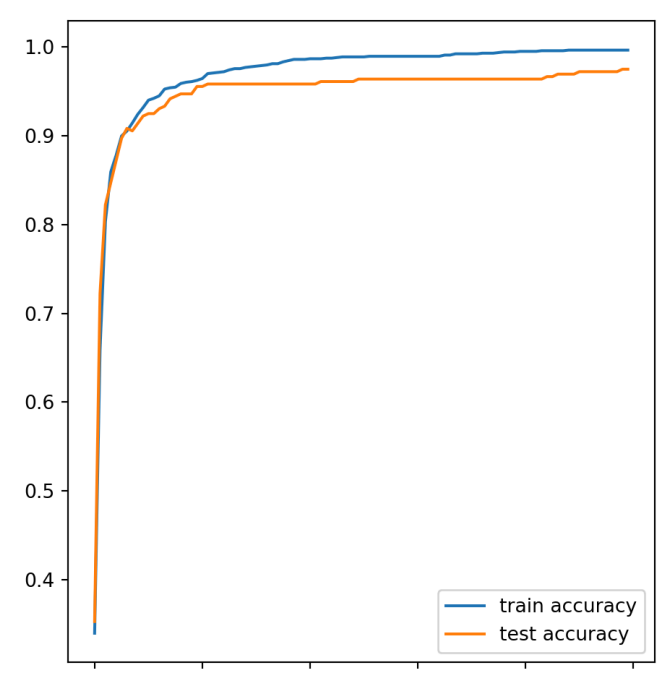
```
train_acc[-5:]
```

```
[0.9965205288796103, 0.9965205288796103, 0.99652
```

```
test_acc[-5:]
```

```
[0.9722222222222222, 0.9722222222222222, 0.97222
```





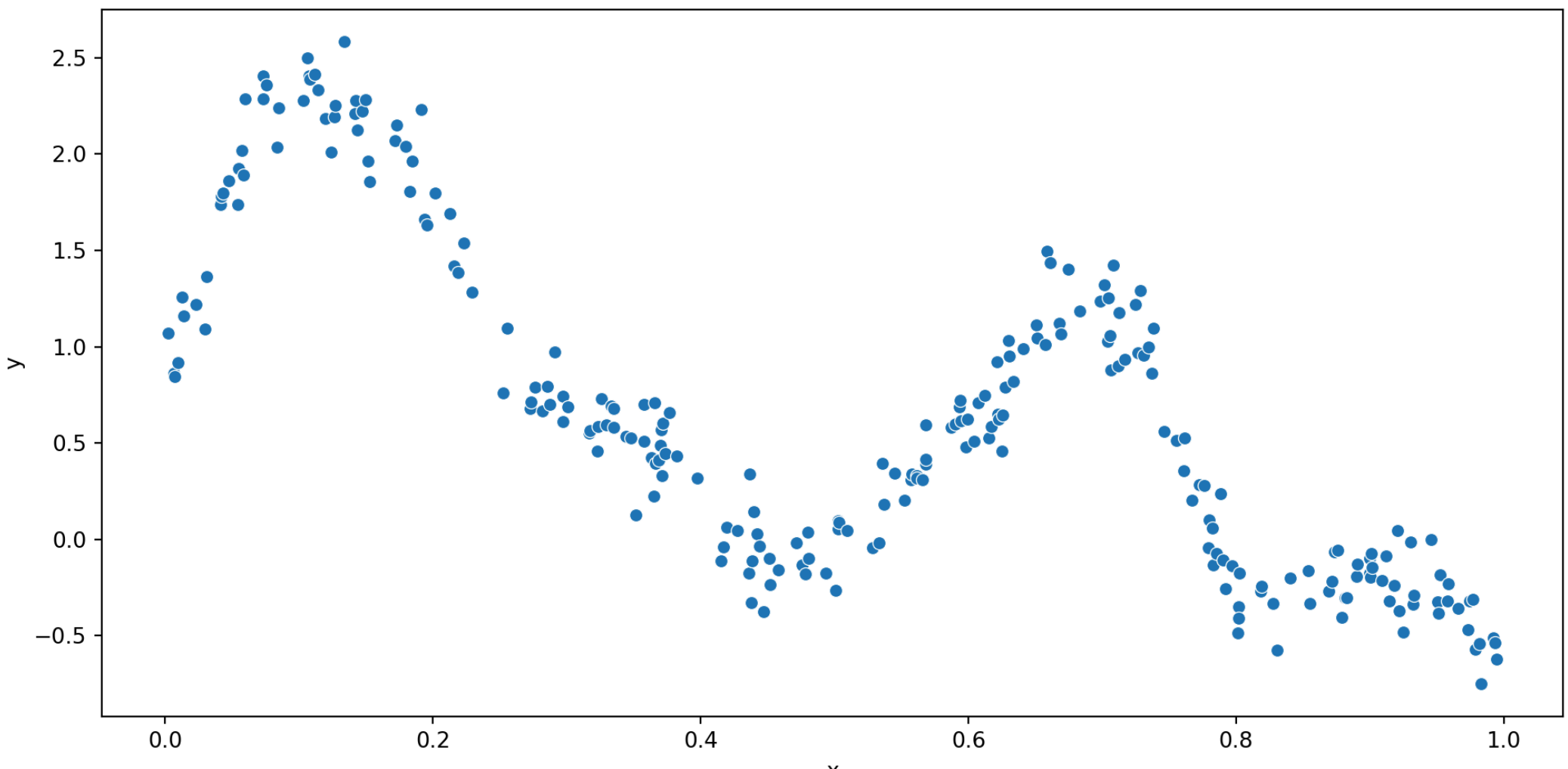
Organizing models

```
class mnist_conv_model2(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.model = torch.nn.Sequential(
          torch.nn.Unflatten(1, (1,8,8)),
          torch.nn.Conv2d(
            in_channels=1, out_channels=8,
            kernel_size=3, stride=1, padding=1
          ),
          torch.nn.ReLU(),
          torch.nn.MaxPool2d(kernel_size=2),
          torch.nn.Flatten(),
          torch.nn.Linear(8 * 4 * 4, 10)
        )

    def forward(self, X):
        return self.model(X)

    def fit(self, X_train, y_train, X_test, y_test, lr=0.001, n=1000, acc_step=10):
      opt = torch.optim.SGD(self.parameters(), lr=lr, momentum=0.9)
      losses, train_acc, test_acc = [], [], []

```

A bit more on non-linear activation layers

Non-linear functions

```python
df = pd.read_csv("data/gp.csv")
X = torch.tensor(df["x"], dtype=torch.float32).reshape(-1,1)
y = torch.tensor(df["y"], dtype=torch.float32)
```



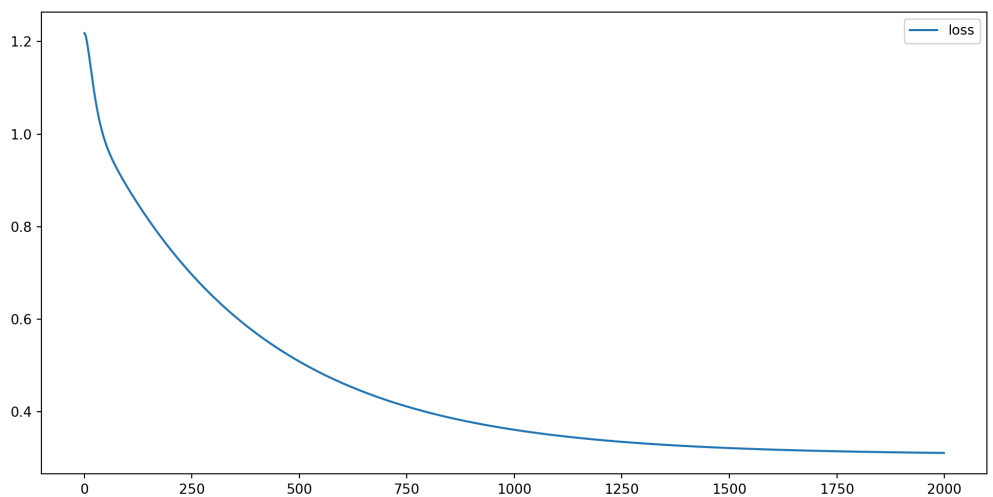
Linear regression

```python
class lin_reg(torch.nn.Module):
    def __init__(self, X):
        super().__init__()
        self.n = X.shape[0]
        self.p = X.shape[1]
        self.model = torch.nn.Sequential(
          torch.nn.Linear(self.p, self.p)
        )

    def forward(self, X):
        return self.model(X)

    def fit(self, X, y, n=1000):
      losses = []
      opt = torch.optim.SGD(self.parameters(), lr=0.001, momentum=0.9)
      for i in range(n):
          loss = torch.nn.MSELoss()(self(X).squeeze(), y)
          loss.backward()
          opt.step()
          opt.zero_grad()
          losses.append(loss.item())

```

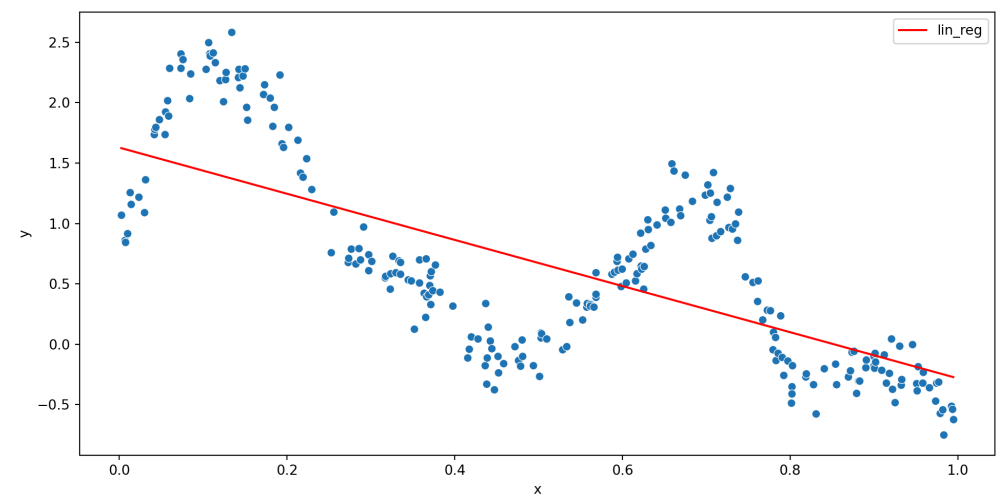
Model results

```
m1 = lin_reg(X)
loss = m1.fit(X,y, n=2000)
```

Training loss:



Predictions



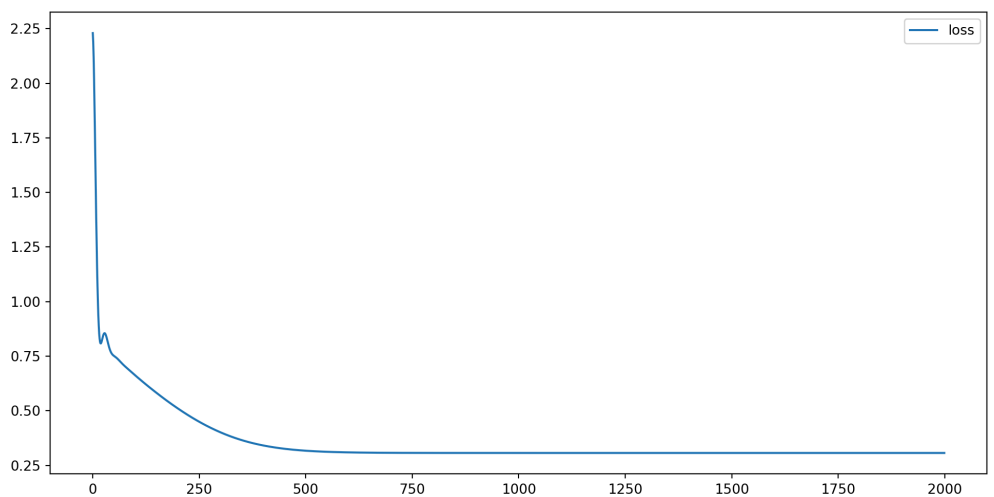
Double linear regression

```python
class dbl_lin_reg(torch.nn.Module):
    def __init__(self, X, hidden_dim=10):
        super().__init__()
        self.n = X.shape[0]
        self.p = X.shape[1]
        self.model = torch.nn.Sequential(
          torch.nn.Linear(self.p, hidden_dim),
          torch.nn.Linear(hidden_dim, 1)
        )

    def forward(self, X):
        return self.model(X)

    def fit(self, X, y, n=1000):
      losses = []
      opt = torch.optim.SGD(self.parameters(), lr=0.001, momentum=0.9)
      for i in range(n):
          loss = torch.nn.MSELoss()(self(X).squeeze(), y)
          loss.backward()
          opt.step()
          opt.zero_grad()
          losses.append(loss.item())
```

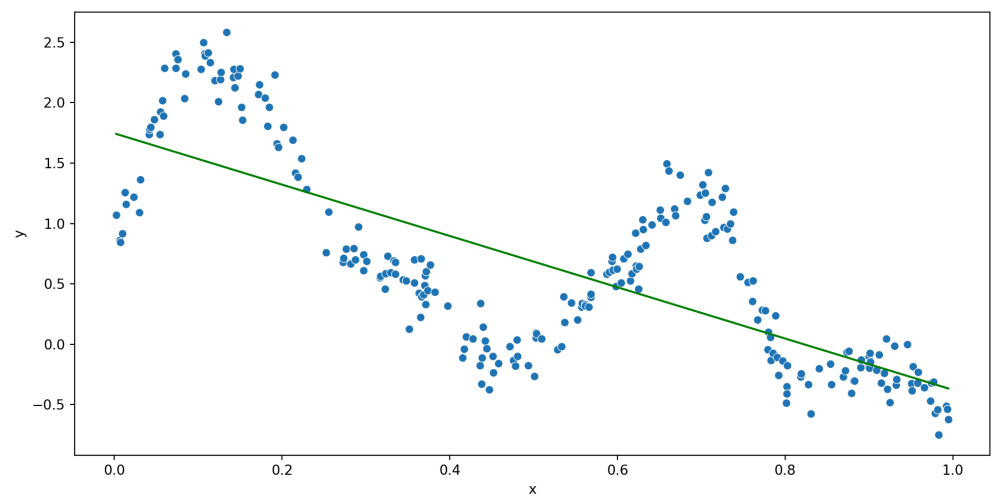
Model results

```
m2 = dbl_lin_reg(X, hidden_dim=10)
loss = m2.fit(X,y, n=2000)
```

Training loss:



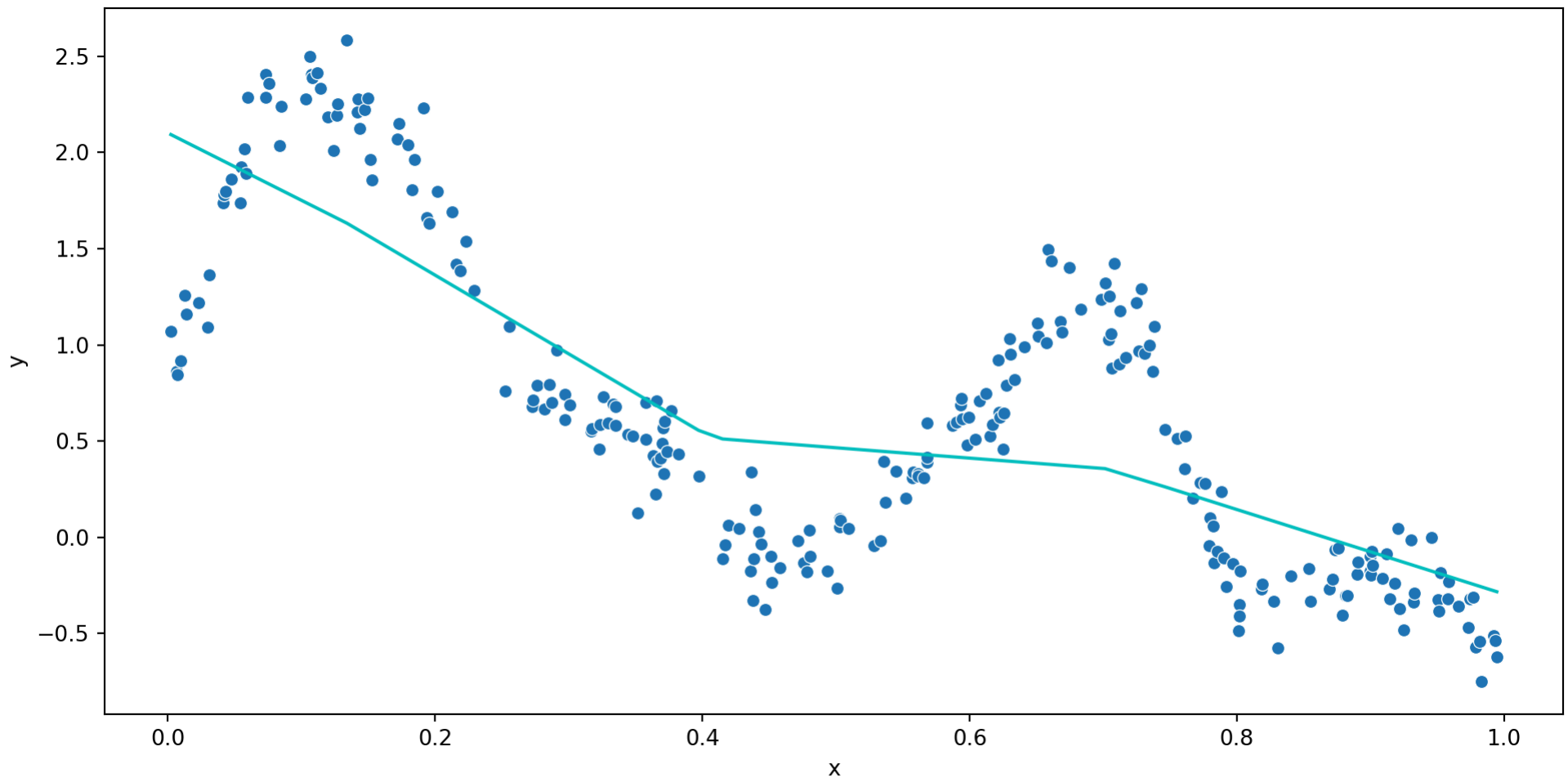
Predictions



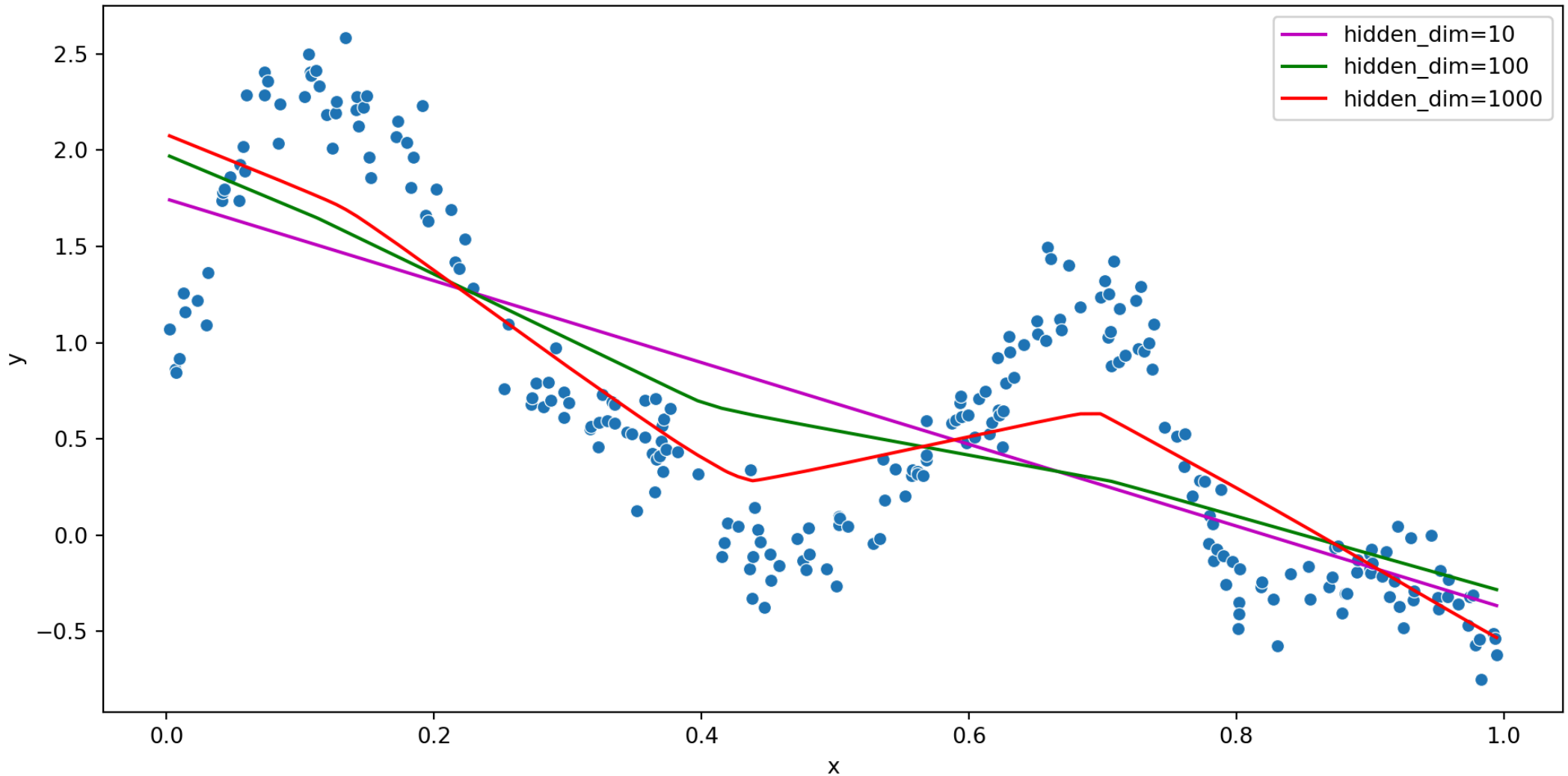
Non-linear regression w/ ReLU

```python
class lin_reg_relu(torch.nn.Module):
    def __init__(self, X, hidden_dim=100):
        super().__init__()
        self.n = X.shape[0]
        self.p = X.shape[1]
        self.model = torch.nn.Sequential(
          torch.nn.Linear(self.p, hidden_dim),
          torch.nn.ReLU(),
          torch.nn.Linear(hidden_dim, 1)
        )

    def forward(self, X):
        return self.model(X)

    def fit(self, X, y, n=1000):
      losses = []
      opt = torch.optim.SGD(self.parameters(), lr=0.001, momentum=0.9)
      for i in range(n):
          loss = torch.nn.MSELoss()(self(X).squeeze(), y)
          loss.backward()
          opt.step()
          opt.zero_grad()
```

Model results



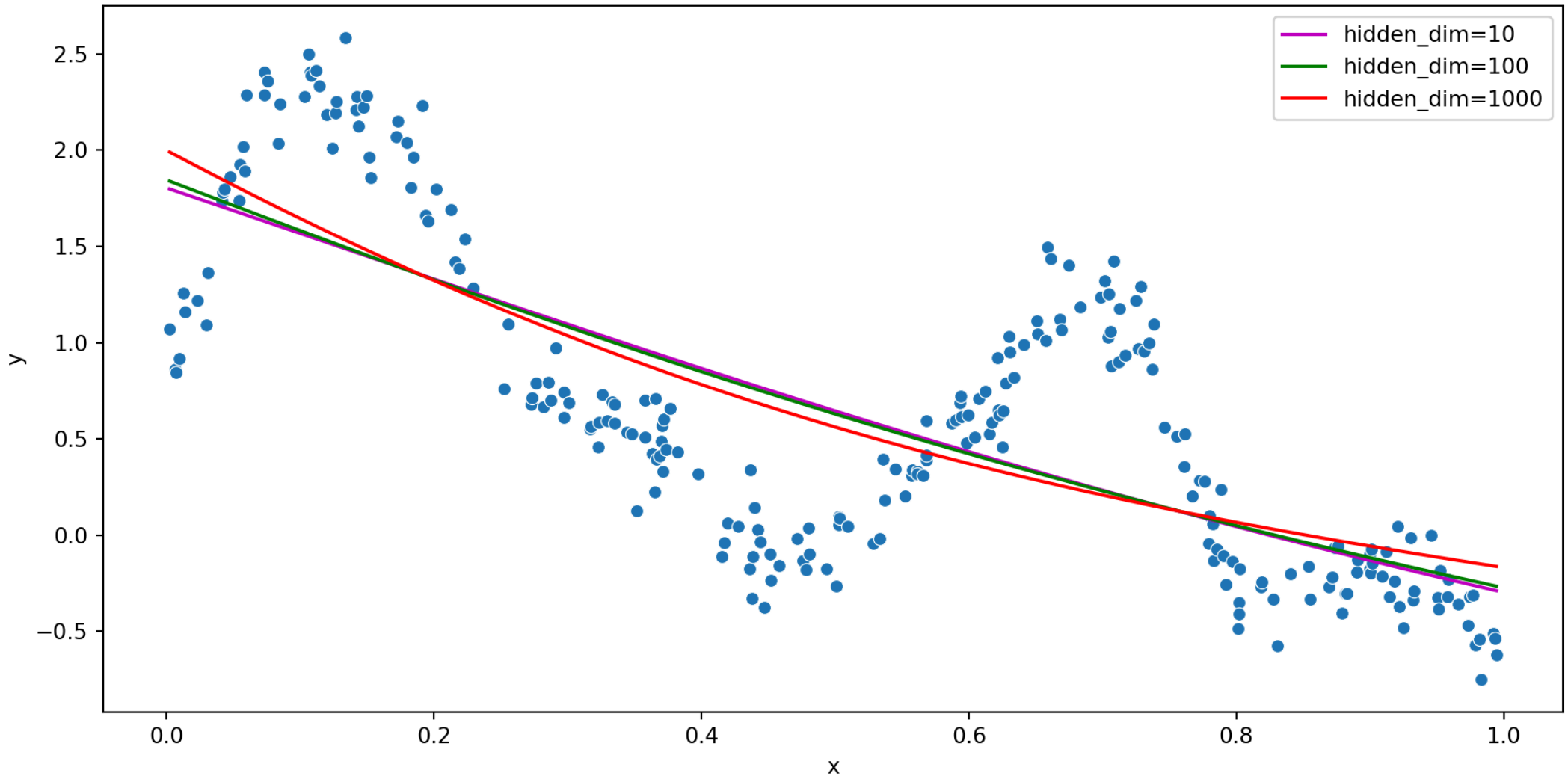
Hidden dimensions



Non-linear regression w/ Tanh

```python
class lin_reg_tanh(torch.nn.Module):
    def __init__(self, X, hidden_dim=10):
        super().__init__()
        self.n = X.shape[0]
        self.p = X.shape[1]
        self.model = torch.nn.Sequential(
          torch.nn.Linear(self.p, hidden_dim),
          torch.nn.Tanh(),
          torch.nn.Linear(hidden_dim, 1)
        )

    def forward(self, X):
        return self.model(X)

    def fit(self, X, y, n=1000):
      losses = []
      opt = torch.optim.SGD(self.parameters(), lr=0.001, momentum=0.9)
      for i in range(n):
          loss = torch.nn.MSELoss()(self(X).squeeze(), y)
          loss.backward()
          opt.step()
          opt.zero_grad()
```

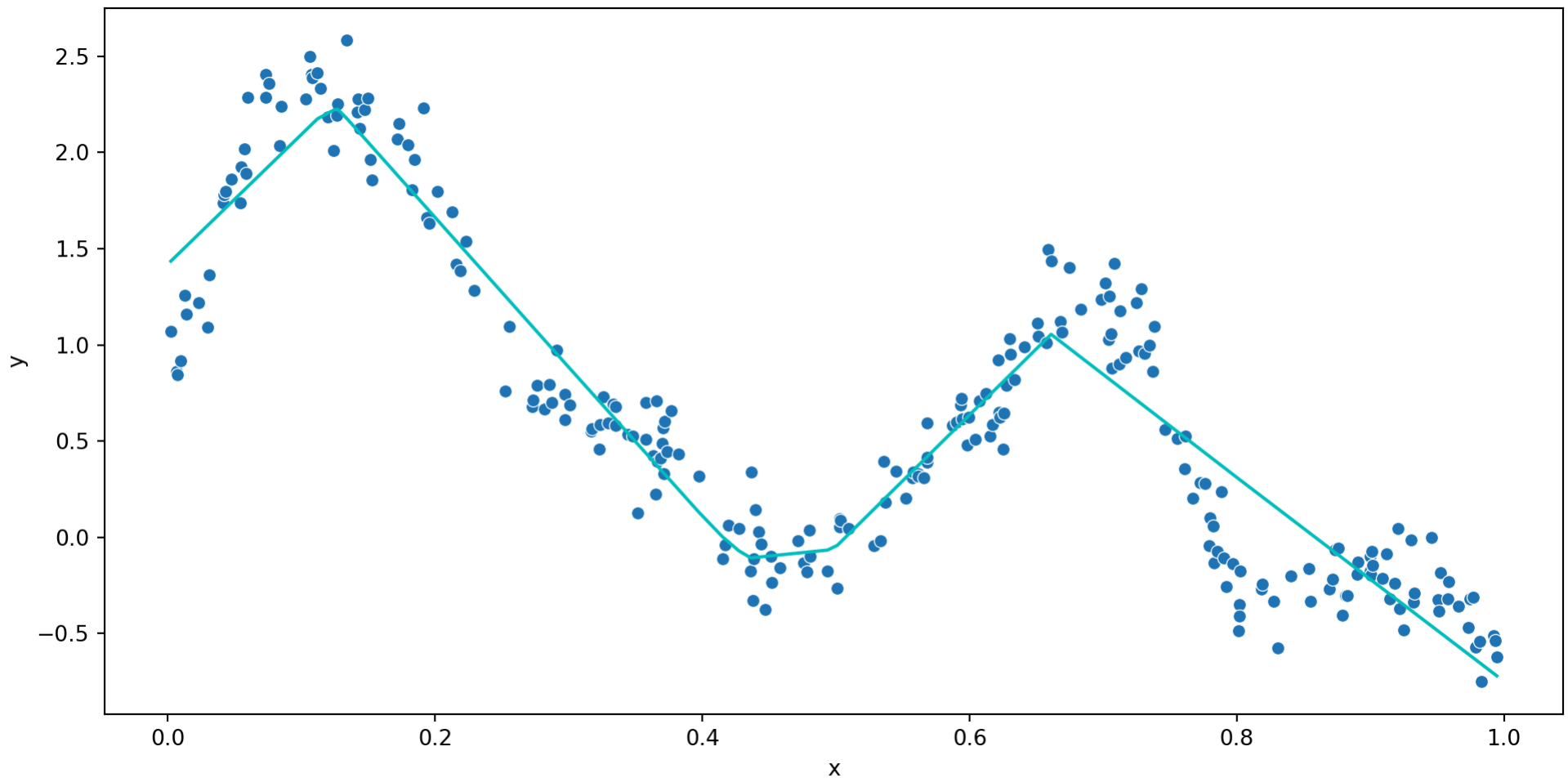
Tanh & hidden dimension



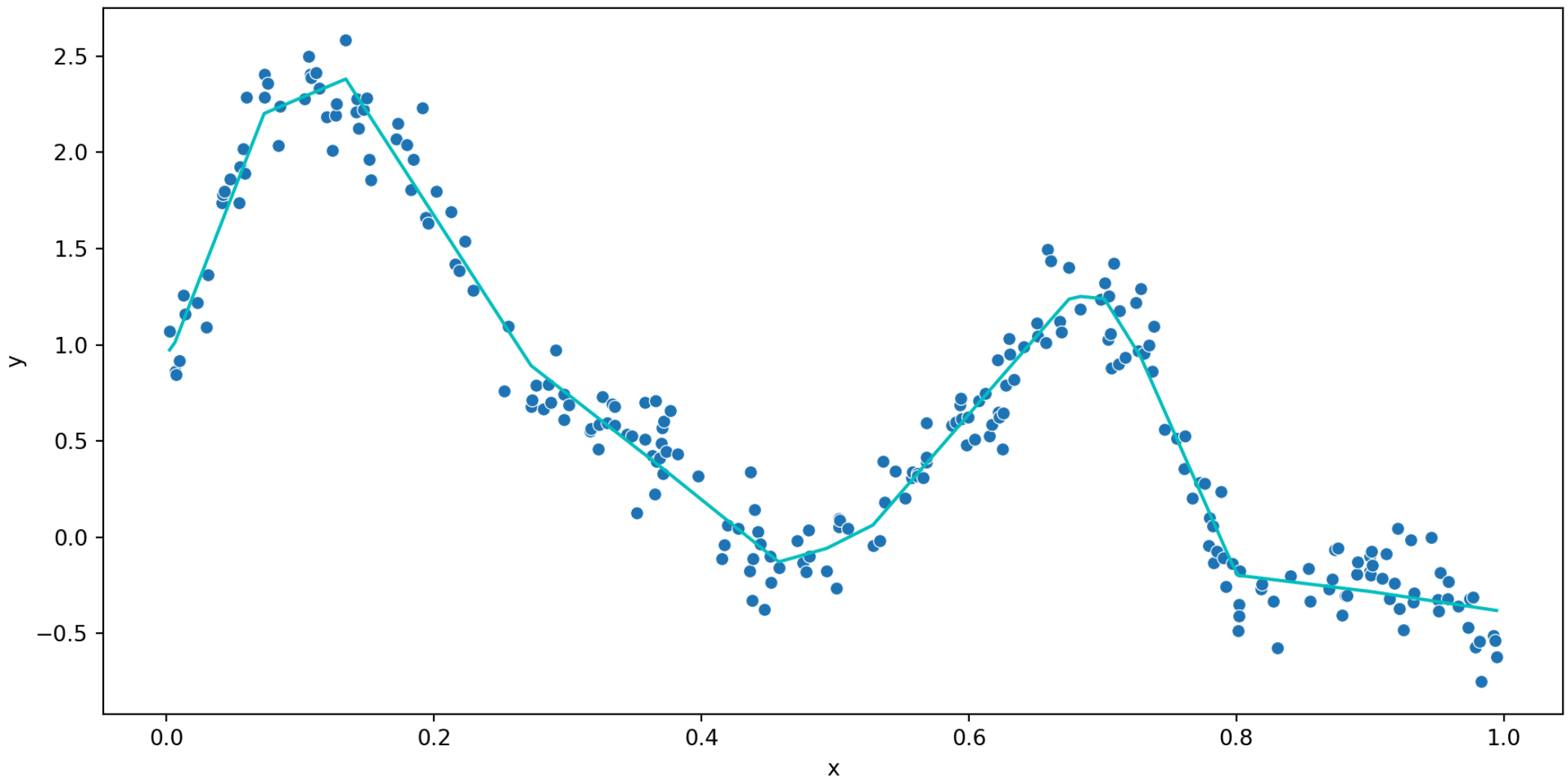
Three layers

```python
class three_layers(torch.nn.Module):
    def __init__(self, X, hidden_dim=100):
        super().__init__()
        self.n = X.shape[0]
        self.p = X.shape[1]
        self.model = torch.nn.Sequential(
          torch.nn.Linear(self.p, hidden_dim),
          torch.nn.ReLU(),
          torch.nn.Linear(hidden_dim, hidden_dim),
          torch.nn.ReLU(),
          torch.nn.Linear(hidden_dim, 1)
        )

    def forward(self, X):
        return self.model(X)

    def fit(self, X, y, n=1000):
      losses = []
      opt = torch.optim.SGD(self.parameters(), lr=0.001, momentum=0.9)
      for i in range(n):
          loss = torch.nn.MSELoss()(self(X).squeeze(), y)
          loss.backward()
```

Model results



Five layers

```python
class five_layers(torch.nn.Module):
    def __init__(self, X, hidden_dim=100):
        super().__init__()
        self.n = X.shape[0]
        self.p = X.shape[1]
        self.model = torch.nn.Sequential(
          torch.nn.Linear(self.p, hidden_dim),
          torch.nn.ReLU(),
          torch.nn.Linear(hidden_dim, hidden_dim),
          torch.nn.ReLU(),
          torch.nn.Linear(hidden_dim, hidden_dim),
          torch.nn.ReLU(),
          torch.nn.Linear(hidden_dim, hidden_dim),
          torch.nn.ReLU(),
          torch.nn.Linear(hidden_dim, 1)
        )

    def forward(self, X):
        return self.model(X)

    def fit(self, X, y, n=1000):
      losses = []
```

Model results